

Student 13 **Jayaraman Renuka Devi** 192573007RD

5 / 5 submitted

Q1. Smart Document Editor using String, StringBuilder and StringBuffer

CODE

```
import java.util.*;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String paragraph = sc.nextLine();
        int characters = paragraph.length();
        String[] words = paragraph.trim().split("\\s+");
        int wordCount = words.length;
        String[] sentences = paragraph.split("[.!?]+");
        int sentenceCount = 0;
        for (String s : sentences) {
            if (!s.trim().isEmpty()) {
                sentenceCount++;
            }
        }
        String oldWord = sc.nextLine();
        String newWord = sc.nextLine();
        String replacedParagraph = paragraph.replace(oldWord, newWord);
        String[] sentenceArray = replacedParagraph.split("(?<=[.!?])");
        StringBuilder finalDocument = new StringBuilder();
        for (String s : sentenceArray) {
            s = s.trim();
            char punctuation = ' ';
            if (s.endsWith(".") || s.endsWith("!") || s.endsWith("?")) {
                punctuation = s.charAt(s.length() - 1);
                s = s.substring(0, s.length() - 1);
            }
            String reversed = new StringBuilder(s).reverse().toString();
            finalDocument.append(reversed);
            if (punctuation != ' ') {
                finalDocument.append(punctuation);
            }
            finalDocument.append(" ");
        }
        StringBuffer history = new StringBuffer();
        history.append("Original Paragraph: ").append(paragraph).append("\n");
        history.append("Replaced \n")
            .append(oldWord)
            .append("\n with \n")
            .append(newWord)
            .append("\n");
        System.out.println("===== DOCUMENT REPORT =====");
        System.out.println("Characters : " + characters);
        System.out.println("Words : " + wordCount);
        System.out.println("Sentences : " + sentenceCount);
        System.out.println("Original Document:");
        System.out.println(paragraph);
        System.out.println("Editing History:");
        System.out.println(history);
        System.out.println("Final Document:");
        System.out.println(finalDocument.toString().trim());

        sc.close();
    }
}
```

INPUT

Java is easy. Java is a programming languages.
Java
Python

OUTPUT

(no output)

Q2. Hospital Token Management System using List Interface

CODE

```
import java.util.*;

class Hospital {
    int patientId;
    String patientName;
    String department;

    void setDetails(int patientId, String patientName, String department) {
        this.patientId = patientId;
        this.patientName = patientName;
        this.department = department;
    }

    void display() {
        System.out.println("Patient ID: " + patientId);
        System.out.println("Patient Name: " + patientName);
        System.out.println("Department: " + department);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        LinkedList<Hospital> list = new LinkedList<>();
        int choice;

        do {
            System.out.println("1.Register Patient");
            System.out.println("2.Display Patients");
            System.out.println("3.Update Patient");
            System.out.println("4.Remove Patient");
            System.out.println("5.Display by Department");
            System.out.println("6.First and Last Patient");
            System.out.println("7.Exit");

            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    Hospital h = new Hospital();
                    int id = sc.nextInt();
                    sc.nextLine();
                    String name = sc.nextLine();
                    String dept = sc.nextLine();
                    h.setDetails(id, name, dept);
                    list.add(h);
                    System.out.println("Patient Registered Successfully.");
                    break;

                case 2:
                    if (list.isEmpty()) {
                        System.out.println("No Patients Registered.");
                    } else {
                        for (Hospital p : list)
                            p.display();
                    }
                    break;

                case 3:
                    int updateId = sc.nextInt();
                    sc.nextLine();
                    boolean updated = false;
```

```
        for (Hospital p : list) {
            if (p.patientId == updateId) {
                p.patientName = sc.nextLine();
                p.department = sc.nextLine();
                updated = true;
                System.out.println("Patient Updated Successfully.");
                break;
            }
        }

        if (!updated)
            System.out.println("Patient Not Found.");
        break;

    case 4:
        int removeId = sc.nextInt();
        Iterator<Hospital> it = list.iterator();
        boolean removed = false;

        while (it.hasNext()) {
            Hospital p = it.next();
            if (p.patientId == removeId) {
                it.remove();
                removed = true;
                System.out.println("Patient Removed Successfully.");
                break;
            }
        }

        if (!removed)
            System.out.println("Patient Not Found.");
        break;

    case 5:
        sc.nextLine();
        String searchDept = sc.nextLine();
        boolean found = false;

        for (Hospital p : list) {
            if (p.department.equalsIgnoreCase(searchDept)) {
                p.display();
                found = true;
            }
        }

        if (!found)
            System.out.println("No Patients Found.");
        break;

    case 6:
        if (list.isEmpty()) {
            System.out.println("No Patients Registered.");
        } else {
            System.out.println("First Registered Patient:");
            list.getFirst().display();
            System.out.println("Last Registered Patient:");
            list.getLast().display();
        }
        break;

    case 7:
        System.out.println("Thank You");
        break;
```

Report

```
        default:
            System.out.println("Invalid Choice");
        }
    } while (choice != 7);

    sc.close();
}
}
```

INPUT

```
1
101
Rahul
Cardiology
7
```

OUTPUT

(no output)

Q3. Airport Boarding Queue Management using Queue Interface

CODE

```
import java.util.*;

public class AirportQueue {
    static Queue<String> queue = new LinkedList<String>();
    static Queue<String> vipQueue = new PriorityQueue<String>();
    static void addPassenger(String name) {
        queue.offer(name);
    }
    static void addVIP(String name) {
        vipQueue.offer(name);
    }
    static void displayQueue() {
        System.out.println("Current Boarding Queue:");
        int i = 1;
        for (String s : queue) {
            System.out.println(i + ". " + s);
            i++;
        }
    }
    static void nextPassenger() {
        if (queue.isEmpty()) {
            System.out.println("Queue is Empty");
        } else {
            System.out.println("Next Passenger: " + queue.peek());
        }
    }
    static void boardPassenger() {
        if (queue.isEmpty()) {
            System.out.println("Queue is Empty");
        } else {
            System.out.println("Boarded Passenger: " + queue.poll());
        }
    }
    static void remainingPassengers() {
        System.out.println("Remaining Passengers");
        int i = 1;
        for (String s : queue) {
            System.out.println(i + ". " + s);
            i++;
        }
    }
    static void displayVIP() {
        System.out.println("VIP Passengers");
        int i = 1;
        for (String s : vipQueue) {
            System.out.println(i + ". " + s);
            i++;
        }
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    int n = sc.nextInt();
    for (int i = 0; i < n; i++) {
        String name = sc.next();
        addPassenger(name);
    }
    displayQueue();
    nextPassenger();
    boardPassenger();
    remainingPassengers();
    int v = sc.nextInt();
    for (int i = 0; i < v; i++) {
```

Report

```
        String name = sc.next();
        addVIP(name);
    }
    displayVIP();
    sc.close();
}
}
```

INPUT

```
5
Arjun
Nisha
Rahul
Priya
Kiran
2
Ramesh
Suresh
```

OUTPUT

(no output)

Q4. Digital Certificate Verification using Set Interface

CODE

```
import java.util.*;

public class CertificateVerification {
    LinkedHashSet<String> certificates = new LinkedHashSet<String>();
    void addCertificate(String id) {
        if (certificates.add(id))
            System.out.println(id + " Added");
        else
            System.out.println(id + " Duplicate Certificate");
    }
    void verifyCertificate(String id) {
        if (certificates.contains(id))
            System.out.println("Certificate Found");
        else
            System.out.println("Certificate Not Found");
    }
    void displayInsertionOrder() {
        System.out.println("Certificates in Insertion Order:");
        for (String id : certificates) {
            System.out.println(id);
        }
    }
    void displaySortedOrder() {
        TreeSet<String> sorted = new TreeSet<String>(certificates);
        System.out.println("Certificates in Sorted Order:");
        for (String id : sorted) {
            System.out.println(id);
        }
    }
    void totalCertificates() {
        System.out.println("Total Unique Certificates: " + certificates.size());
    }
    void removeCertificate(String id) {
        if (certificates.remove(id))
            System.out.println("Certificate Removed");
        else
            System.out.println("Certificate Not Found");
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        CertificateVerification obj = new CertificateVerification();
        int n = sc.nextInt();
        for (int i = 0; i < n; i++) {
            String id = sc.next();
            obj.addCertificate(id);
        }
        obj.displayInsertionOrder();
        String verifyId = sc.next();
        obj.verifyCertificate(verifyId);
        obj.displaySortedOrder();
        obj.totalCertificates();
        String removeId = sc.next();
        obj.removeCertificate(removeId);
        System.out.println("Final Certificate List:");
        obj.displayInsertionOrder();
    }
}
```

INPUT

```
7
CERT101
CERT205
CERT110
CERT305
```



```
CERT205  
CERT450  
CERT550  
CERT110  
CERT205
```

OUTPUT

```
(no output)
```

Q5. Smart Inventory Management using Map Interface

CODE

```
import java.util.*;

public class Inventory {
    HashMap<Integer, Product> products = new HashMap<Integer, Product>();
    class Product {
        int id;
        String name;
        String category;
        int quantity;
        double price;
        Product(int id, String name, String category, int quantity, double price) {
            this.id = id;
            this.name = name;
            this.category = category;
            this.quantity = quantity;
            this.price = price;
        }
    }
    void addProduct(int id, String name, String category, int quantity, double price) {
        products.put(id, new Product(id, name, category, quantity, price));
    }
    void updateQuantity(int id, int quantity) {
        if (products.containsKey(id)) {
            products.get(id).quantity = quantity;
            System.out.println("Quantity Updated");
        } else {
            System.out.println("Product Not Found");
        }
    }
    void searchProduct(int id) {
        if (products.containsKey(id)) {
            Product p = products.get(id);
            System.out.println("Product ID : " + p.id);
            System.out.println("Name : " + p.name);
            System.out.println("Category : " + p.category);
            System.out.println("Quantity : " + p.quantity);
            System.out.println("Price : " + p.price);
        } else {
            System.out.println("Product Not Found");
        }
    }
    void totalValue() {
        double total = 0;
        for (Product p : products.values()) {
            total += p.quantity * p.price;
        }
        System.out.println("Total Inventory Value = " + total);
    }
    void displayCategory(String category) {
        System.out.println("Products in " + category + " Category");
        for (Product p : products.values()) {
            if (p.category.equalsIgnoreCase(category)) {
                System.out.println(p.id + " " + p.name + " " + p.quantity);
            }
        }
    }
    void removeProduct(int id) {
        if (products.remove(id) != null)
            System.out.println("Product Removed");
        else
            System.out.println("Product Not Found");
    }
}
```

Report

```
void displayProducts() {
    System.out.println("Available Products");
    for (Product p : products.values()) {
        System.out.println(p.id + " " + p.name + " " + p.category + " " + p.quantity + " " + p.price);
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    Inventory obj = new Inventory();
    int n = sc.nextInt();
    for (int i = 0; i < n; i++) {
        int id = sc.nextInt();
        String name = sc.next();
        String category = sc.next();
        int quantity = sc.nextInt();
        double price = sc.nextDouble();
        obj.addProduct(id, name, category, quantity, price);
    }
    int updateId = sc.nextInt();
    int newQty = sc.nextInt();
    obj.updateQuantity(updateId, newQty);
    int searchId = sc.nextInt();
    obj.searchProduct(searchId);
    obj.totalValue();
    String category = sc.next();
    obj.displayCategory(category);
    int removeId = sc.nextInt();
    obj.removeProduct(removeId);
    obj.displayProducts();
    sc.close();
}
}
```

INPUT

```
5
301 Football Sports 15 1200
302 CricketBat Sports 10 2500
303 TennisBall Sports 40 150
304 Laptop Electronics 8 55000
305 Mouse Electronics 20 700
302 25
304
Sports
303
```

OUTPUT

(no output)